

Reviewer Pack — Minimal p-Adic Hodge Class and Communication Complexity Rank...

Entry b8f0f59832d0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 09:50:18 UTC

Minimal p-Adic Hodge Class and Communication Complexity Rank Correlation

Entry ID: b8f0f59832d0 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 09:50:18 UTC

1. Conjecture

Field A (mathematical branch): p-adic Number Theory (Hodge theory) **Field B** (complexity object): Communication Complexity

Statement:

For every Boolean function f with n variables, the minimal p-adic Hodge class $h(f)$ of its associated algebraic variety is linearly correlated with its communication complexity rank $r(f)$, such that $\log(h(f)) = \Theta(r(f))$.

Rationale (proposer's reasoning):

p-adic Hodge theory provides a framework for studying the arithmetic nature of varieties over finite fields, which may expose subtle structures in Boolean functions affecting their communication complexity. The Hodge class is computable using the Hasse-Weil L-function and may serve as an invariant that captures the depth of the communication channel.

Taxonomy category: p-adic_Hodge_theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 281a0ac39a44765d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a Boolean function f with $n \leq 40$ variables, the conjecture is supported if at least 80% of 30 random seeds yield a correlation coefficient between $\log(h(f))$ and $r(f)$ within ± 3 standard deviations from their expected value.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "p-adic Hodge class" AND "communication complexity rank" - "Hodge theory" AND "communication complexity" - "algebraic variety" inauthor:"communication complexity"

Top relevant hits considered: - [http://arxiv.org/abs/2210.01601v2] Quantum communication complexity of linear regression - [http://arxiv.org/abs/1403.8106v1] Recent advances on the log-rank conjecture in communication complexity - [http://arxiv.org/abs/0909.5684v1] Partition Arguments in Multiparty Communication Complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=241.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_complexity_rank(f):
        n = len(f)
        if n == 1:
            return 1
        else:
            # Simplified rank calculation for demonstration purposes
            return n - 1

    def p_adic_hodge_class(f):
        # Placeholder function to simulate the computation of the p-adic Hodge class
        # This is a dummy implementation and should be replaced with actual computation
        return random.random()

    results = []
    for _ in range(30):
        n = random.choice([5, 10, 15, 20, 30, 40])
        f = generate_boolean_function(n)
        h_f = p_adic_hodge_class(f)
        r_f = communication_complexity_rank(f)
```

```

    if h_f <= 0 or r_f <= 0:
        continue

    log_h_f = math.log(h_f)
    results.append((log_h_f, r_f))

if not results:
    return {
        "metric_name": "correlation_coefficient",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "empty_results"
    }

log_h_f_values, r_f_values = zip(*results)
mean_log_h_f = sum(log_h_f_values) / len(log_h_f_values)
mean_r_f = sum(r_f_values) / len(r_f_values)
std_log_h_f = math.sqrt(sum((x - mean_log_h_f)**2 for x in log_h_f_values) / len(log_h_f_values))
std_r_f = math.sqrt(sum((x - mean_r_f)**2 for x in r_f_values) / len(r_f_values))

correlation_coefficient = sum((log_h_f_values[i] - mean_log_h_f) * (r_f_values[i] - mean_r_f)

return {
    "metric_name": "correlation_coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": max(n for _ in range(30)),
    "conjecture_holds": abs(correlation_coefficient) >= 3 * std_correlation_coefficient,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [random.randint(2, 10**9) for _ in range(30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(x["metric_value"] for x in results if x["metric_value"] is not None) /
    std_metric_value = math.sqrt(sum((x["metric_value"] - mean_metric_value)**2 for x in results if
    support_fraction = sum(x["conjecture_holds"] for x in results) / len(results)

    if all(x["conjecture_holds"] for x in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction=
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction=
    else:
        for result in results:
            if not result["conjecture_holds"]:
                print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_fail
                break

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not meet the pre-registered support condition for the conjecture. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	27836
2	preregistration	ollama_remote	glm4:latest	0	0	16444
3	novelty	ollama_remote	glm4:latest	0	0	8461
4	novelty	ollama_remote	glm4:latest	0	0	14684
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17767
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12717
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	30319
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	38507
9	judge	ollama_remote	glm4:latest	0	0	29641

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 196375 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/b8f0f59832d0.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b8f0f59832d0.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b8f0f59832d0.tar.gz` (if generated)